



SCHOOL OF MATHEMATICS AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING

TO : MR. S. YUTE

FROM : DEBORAH CHIRAMBO & SHALOM MAKWINJA

REGISTRATION NO : BSC-COM-NE-19-21 & BSC-COM-NE-19-22

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION TO
DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE: IoT Telemetry Pipeline & Dashboard

DUE DATE : 9th MAY, 2026.

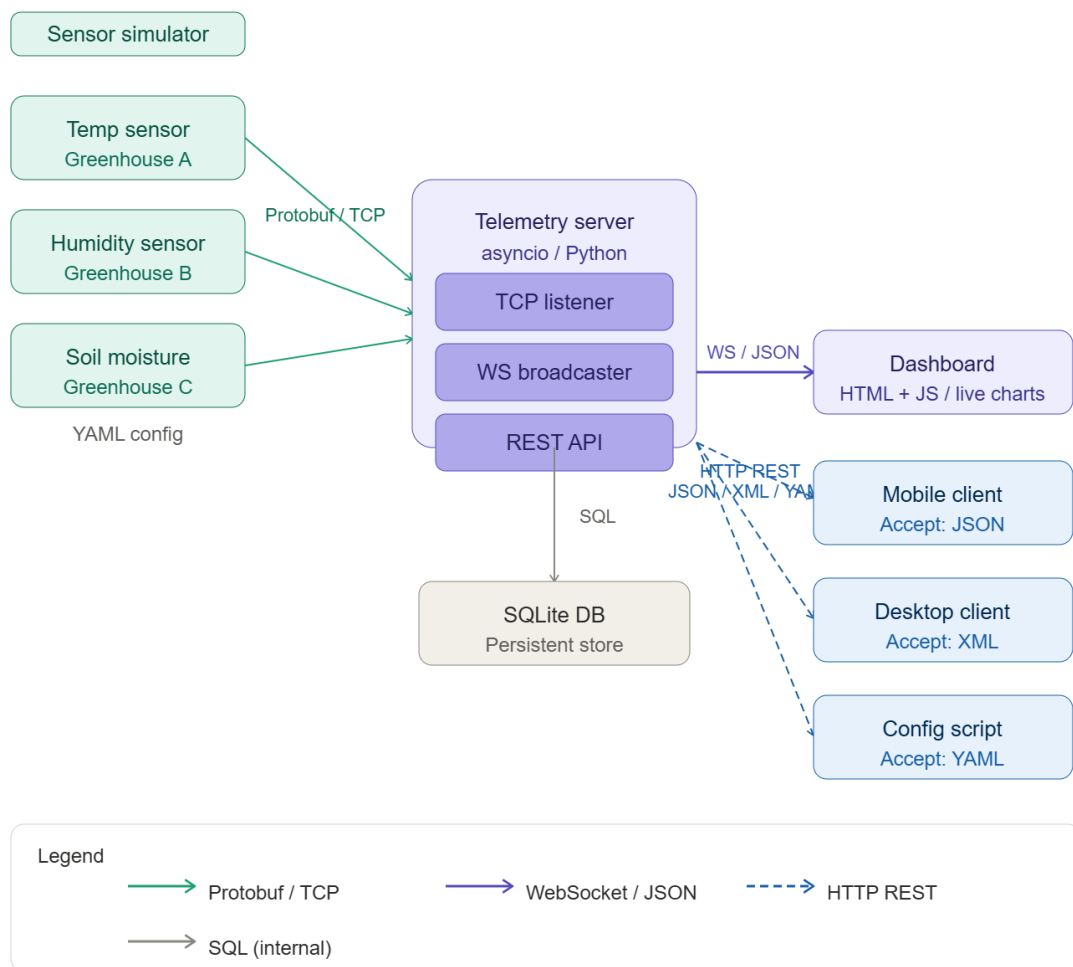


Figure 1 — System Architecture Diagram

The diagram illustrates all major components of the IoT telemetry pipeline and the protocol or serialization format used on every communication link.

Component descriptions:

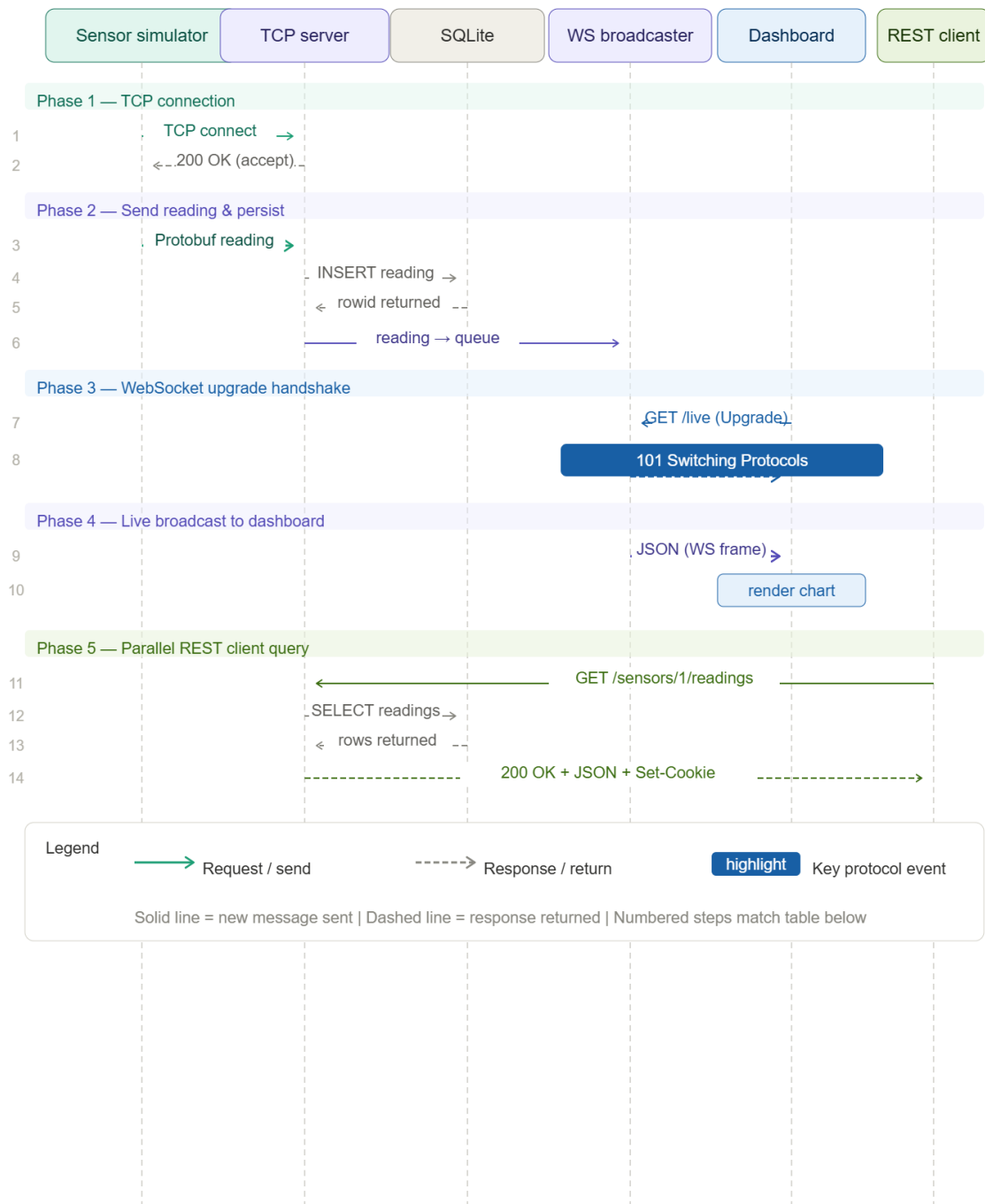
Component	Technology	Role
Sensor Simulator	Python asynco	Generates sensor readings and pushes them to the server via Protobuf-encoded TCP messages
Telemetry Server	aiohttp / asynco	Accepts TCP connections, routes data to storage, and broadcasts live readings to WebSocket clients

SQLite DB	aiosqlite	Persistent storage for all sensor registrations and time-series readings
REST API	aiohttp routes	Handles historical queries with content negotiation — supports JSON, XML, and YAML via the Accept header; assigns session cookies
WebSocket Feed	aiohttp WS	Pushes live readings to subscribed clients over the /live endpoint
Dashboard	HTML + JS	Single-page application that connects to /live and plots real-time charts

Protocol links explained:

- **Protobuf / TCP** — Sensors connect persistently to the telemetry server and stream binary Protobuf-encoded [SensorReading](#) messages. Protobuf is chosen for compactness and schema enforcement.
- **SQL** — The telemetry server uses aiosqlite to perform async INSERT and SELECT operations against the local SQLite database without blocking the event loop.
- **WebSocket / JSON** — The dashboard subscribes to the [/live](#) endpoint. The server broadcasts each new reading as a JSON frame over the established WebSocket connection.
- **HTTP REST (JSON / XML / YAML)** — REST clients query historical data. The server performs content negotiation based on the [Accept](#) header, returning the same data in the client's preferred format.

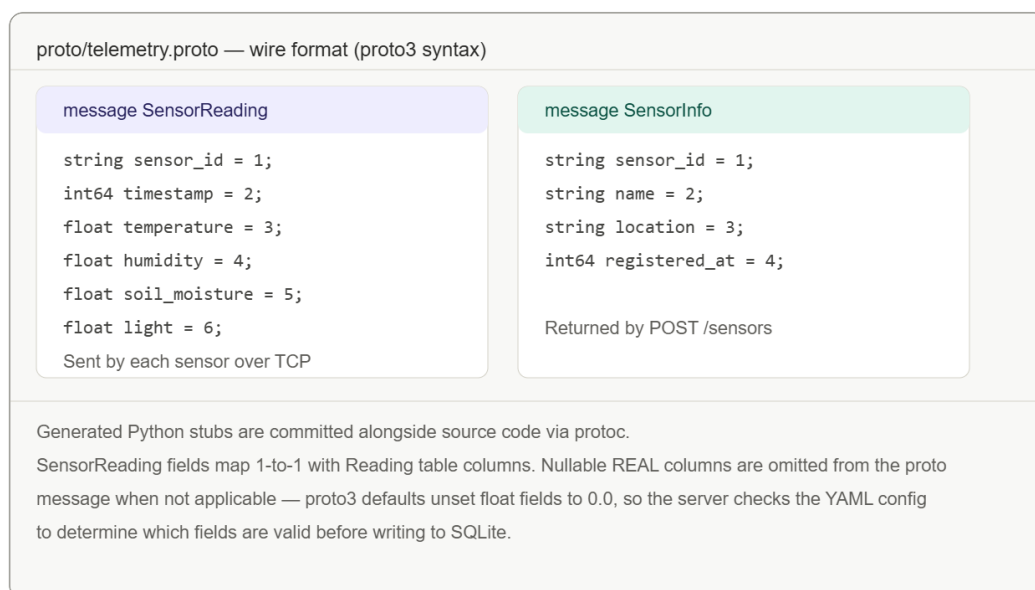
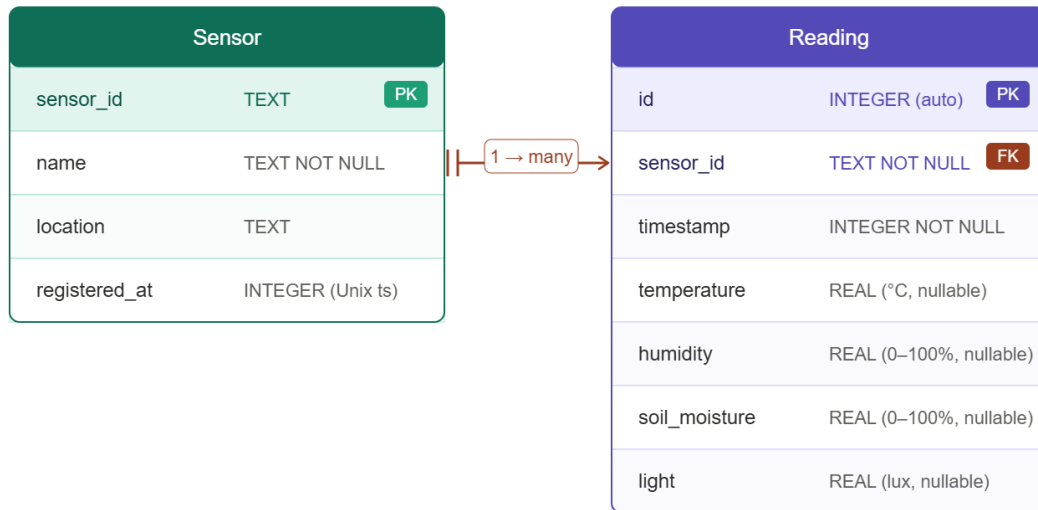
Sequence diagram



Clean sequence diagram covers across 5 phases:

- **Phase 1** — TCP connect and server accept
- **Phase 2** — Protobuf reading sent, INSERT to SQLite, rowid confirmed, reading pushed to queue
- **Phase 3** — WebSocket upgrade handshake with the **101 Switching Protocols** highlighted in blue (the marker the assignment specifically asks for)
- **Phase 4** — Live JSON frame broadcast, dashboard renders chart
- **Phase 5** — Parallel REST client query, SQLite SELECT, 200 OK with Set-Cookie response

Data model



Entity Schemas (SQLite)

Two entities are persisted in SQLite: **Sensor** and **Reading**, linked by a one-to-many relationship — one sensor produces many readings.

The **Sensor** table is a registry. Its primary key **sensor_id** matches the ID defined in the YAML config, ensuring consistency across all system layers. It also stores a human-readable **name**, a **location** string, and a **registered_at** Unix timestamp.

The **Reading** table stores all time-series measurements. It references **Sensor** via a **sensor_id** foreign key. The four measurement columns (**temperature**, **humidity**, **soil_moisture**, **light**) are all nullable — a sensor only populates the fields it actually measures, as defined in its YAML config. Storing **NULL** rather than zero makes it clear when a value is absent versus genuinely zero.

Protobuf Wire Format

Sensor-to-server communication uses binary Protobuf encoding defined in **proto/telemetry.proto**. Protobuf was chosen over JSON for two reasons: smaller payloads over TCP, and strict schema enforcement that catches malformed messages immediately at deserialization.

SensorReading is sent by each sensor over TCP — its six fields map directly to the **Reading** table columns. **SensorInfo** is returned by **POST /sensors** to confirm a newly registered sensor. Python stubs are generated via **protoc** and shared by both the simulator and server.

REST endpoint specification

Method	Path	Request body	Response body	Status codes
GET	/sensors Returns all registered sensors. Cookie set if not already present.	None	[{sensor_id, name, location}]	200
Example: GET /sensors Accept: application/json → 200 OK [{sensor_id:"s1", name:"Temp A", location:"GH-1"}]				
GET	/sensors/{id}/readings Query params: ?from=&to= (optional) 404 if sensor unknown.	None	[{id, sensor_id, timestamp, temperature, humidity, ...}]	200 404 400
Example: GET /sensors/s1/readings?from=1715000000&to=1715086400 Accept: application/xml → 200 OK (XML body)				
POST	/sensors Registers a new sensor. 409 if sensor_id already exists.	{sensor_id, name, location}	{sensor_id, name, location}	201 400 409
Example: POST /sensors Body: {sensor_id:"s4", name:"Light D", location:"GH-4"} → 201 Created				
DELETE	/sensors/{id} Deletes sensor and all associated readings. 404 if sensor unknown.	None	Empty body	204 404
Example: DELETE /sensors/s4 → 204 No Content				
Content negotiation — all endpoints All endpoints support server-driven content negotiation via the HTTP Accept header. Supported types: application/json (default) application/xml application/x-yaml 406 Not Acceptable if unsupported.				
Session cookie A session_id cookie is set on the first request and read on all subsequent calls to identify the client.				

REST Endpoint Specification

The system exposes four REST endpoints, all served by the [aiohttp](#) telemetry server. Every endpoint supports content negotiation via the HTTP [Accept](#) header — the same data can be returned as JSON,

XML, or YAML depending on what the client requests. A `session_id` cookie is assigned on the first request and read on all subsequent calls to identify the client.

`GET /sensors` returns the full list of registered sensors. `GET /sensors/{id}/readings` retrieves historical readings for a specific sensor, with optional `from` and `to` query parameters to filter by time range.

`POST /sensors` registers a new sensor, returning `409 Conflict` if the `sensor_id` already exists. `DELETE /sensors/{id}` removes a sensor and cascades deletion to all its associated readings.

5. YAML Configuration Schema

The sensor simulator reads sensor settings from the `config/sensors.yaml` file. All sensors are listed under the `sensors` key.

Each sensor includes:

- **id** – unique sensor ID
- **name** – sensor name
- **type** – sensor type (`temperature`, `humidity`, `soil_moisture`, `light`, or `greenhouse`)
- **interval** – reporting time in seconds
- **location** – sensor location (optional)

Example: `config/sensors.yaml`

sensors:

```
- id: "sensor-1"
  name: "Greenhouse 1 Temperature"
  type: "temperature"
  interval: 5
  location: "Blantyre Farm A"

- id: "sensor-2"
  name: "Greenhouse 2 Humidity"
  type: "humidity"
  interval: 10
  location: "Blantyre Farm B"
```

6. Concurrency Model

The system uses a single `asyncio` event loop for all operations. No threads or multiprocessing are used.

Main Coroutines

Coroutine	Function
<code>main()</code>	Starts the TCP server, HTTP server, and WebSocket service.
<code>handle_sensor()</code>	Receives sensor data, deserialises Protobuf messages, stores them in SQLite, and sends readings to the queue.
<code>live_broadcast()</code>	Reads sensor data from the queue and broadcasts it to connected WebSocket clients.
<code>websocket_handler()</code>	Manages WebSocket client connections and subscriptions.
<code>rest_* handlers</code>	Handles REST API requests and returns JSON/XML/YAML responses.

Communication Flow

- Sensors send telemetry through TCP.
- Data is stored asynchronously in SQLite.
- Readings are added to an `asyncio.Queue`.
- WebSocket clients receive live updates from the queue.

Slow Client Handling

If a WebSocket client becomes too slow:

- The server monitors the client queue size.
- When the queue exceeds **500 messages**, the oldest message is removed.
- A warning message is logged.

This prevents slow clients from affecting the overall system performance.

Failure	Handling
Sensor disconnect	The server logs the error, closes the connection safely, and waits for the sensor to reconnect. Previous sensor data remains stored in the database.

Storage failure	Database errors are logged and affected REST requests return 500 Internal Server Error . Live updates continue running normally.
Server overload	If the asyncio.Queue becomes full, new readings are dropped and a warning is logged to prevent system slowdown.
Invalid Protobuf data	The server logs the error and closes the faulty sensor connection immediately.